HoopHub

BookGameSeat

Table of Contents

1 Software Requirement Specification 3

1. Introduction 3
2. User Stories 4
3. Functional Requirements 5
4. Use Cases 5

2 Storyboards 7

3 Design 11

1. Class Diagram 11
2. Design-Level Diagram 12
3. Design Patterns 13
4. Activity Diagram 14
5. Sequence Diagram 15
6. State Diagram 16

4 Testing 16

HoopHub

1 Software Requirement Specification

1. Introduction

1. Aim of the document

This document provides a detailed overview of HoopHub, the software developed to connect NBA fans with the best venues to watch games, through an integrated seat booking and event management system. Additionally, it illustrates the various design phases that led to its development. HoopHub was conceived as an academic project within the Software Engineering and Web Design (ISPW) course, during the 2025/2026 academic year. The project was carried out by student Elia Cinti.

2. Overview of the defined system

The system is designed to support support Fans and Venue Managers, offering an intuitive and functional platform dedicated to the NBA "Watch Party" experience. This version of the application distinguishes between users registered as Fans and Venue Managers, providing role-specific functionalities, and is accessible both through a graphical user interface (JavaFX GUI) and through a command-line interface (CLI).

Venue Managers can:
- Register their venue (pub, bar, sports arena) and provide useful information to present it to users.
- Associate NBA games on the schedule with their venue, indicating which games will be broadcast.
- Manage capacity in real time, monitoring available seats for each event.
- Receive notifications regarding bookings made by Fans.

Fan have access to tools for:
- Entering and updating their personal data and sports preferences (favorite team).
- Searching for venues broadcasting a specific NBA game.
- Booking seats quickly and easily, with the assurance of finding a spot in front of the right screen.
- Receiving booking confirmation notifications and updates on schedule changes.

A distinctive feature of the system is polymorphic persistence: the system supports three data storage modes, interchangeable at runtime (MySQL for robust production environments, CSV for portability and local file-system storage, In-Memory for fast testing). Thanks to the Observer pattern, modifications made on one persistence layer are instantly reflected on the other through bidirectional synchronization, ensuring data consistency.

3. HW e SW requirements

   Software:
   - Draw.io for creating system design diagrams.
   - Scene Builder for generating FXML files used in the graphical interface.
   - Figma for prototypes.
   - IntelliJ IDEA as the development platform.
   - JDK 21 or later.
   - Maven for dependency management and project compilation.
   - MySQL Server for relational database persistence.

   Hardware:
   - Operating Systems:
     - Windows 7 or later.
     - macOS v10.7 or later.
     - Linux distributions with JavaFX support.
   - Processor: Minimum 1 GHz, recommended 2 GHz.
   - Memory (RAM): Minimum 2 GB, recommended 4 GB.
   - Hard Disk: Minimum 3 GB, recommended 5 GB.

4. Related systems (at least 2), Pros and Cons.
   - MatchPint (Fanzo): Leading app for finding bars that broadcast sports.
     - *Pro vs HoopHub:* Vast global database of sports venues with extensive coverage across multiple sports and countries.
     - *Con vs HoopHub:* Lacks integrated booking, often redirecting users to the venue's phone number or external forms. Does not manage venue capacity in real time, offering no guarantee of finding an available seat.

   - TheFork / OpenTable: Leading platforms for restaurant table reservations.
     - *Pro vs HoopHub:* Excellent management of seats, time slots, and availability in restaurants, with a well-established user base and review system.
     - *Con vs HoopHub:* Lacks sports-related logic; they do not know "who is playing tonight." Users must manually check whether the venue has a TV and whether it is broadcasting the desired game.
   - Google Maps: The absolute standard for finding places and reading reviews
     - *Pro vs HoopHub:* Unmatched coverage for locating venues and consulting user reviews, with integrated navigation and real-time information.
     - *Con vs HoopHub:* No information on TV schedules, making it impossible to know with certainty whether a specific NBA game will be broadcast. No sports-dedicated booking management.

2. User Stories

1. As a registered fan, I want to book a seat in a venue for my favorite team’s NBA game, so that I can get a reservation.

2. As a guest user, I want to view the schedule of the next two weeks NBA games, so that I can know when matches will take place.

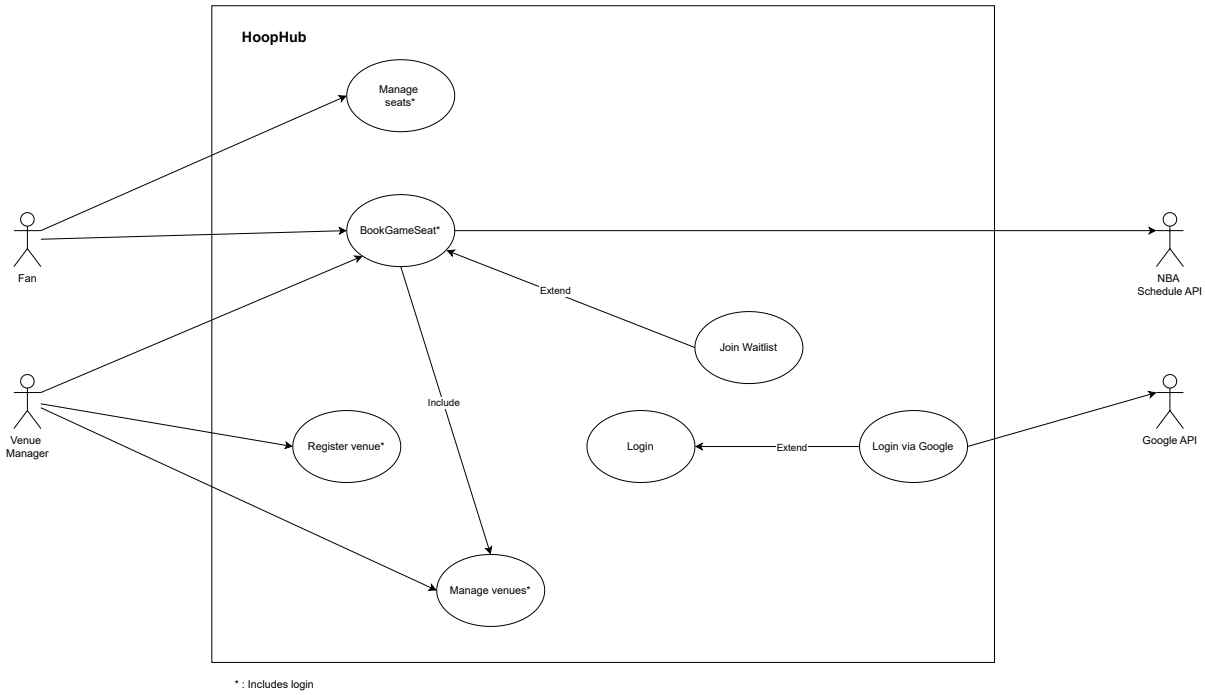
3. As a venue manager, I want to register my venue with name, address, accessibility and available channels, so that fans can discover it and send booking request.

3. Functional Requirements

1. The system shall provide users authentication via username and password.
2. The system shall provide a form to request a seat reservation for an NBA game in a venue.
3. The system shall provide a form to register a new venue.

4. Use Cases

1. Use Case Diagram



2. Internal Steps – BookGameSeat

Precondition: Fan / Venue Manager is logged in.

Main Succes Scenario

1. The fan requests to book a seat for an NBA game.
2. The system retrieves the game from the NBA Schedule API.
3. The system displays the list of NBA games schedule*.
4. The fan selects a specific game.
5. The system retrieves and displays available venues for that game.
6. The fan selects a venue.
7. The system displays the venue details* and requests confirmation.
8. The fan confirms the booking request.
9. The system registers the booking request with status* pending and generates a pre-confirmation code*.
10. The system notifies* the Venue Manager about the new booking request.
11. The Venue Manager reviews and accepts the booking request.
12. The system updates the request status* to confirmed and notifies* the Fan.

Extensions (Alternative Scenario)

3a. *No games available*: The system informs the Fan that no upcoming games are currently available. Use case ends.

5a. *No venues available for the selected game*: The System informs the fan that no venues are currently offering that game and suggests choosing another game. Use case resumes at step 3.

6a. *Fan applies filters to refine the venue list*: The System updates and displays the filtered results based on the selected criteria.

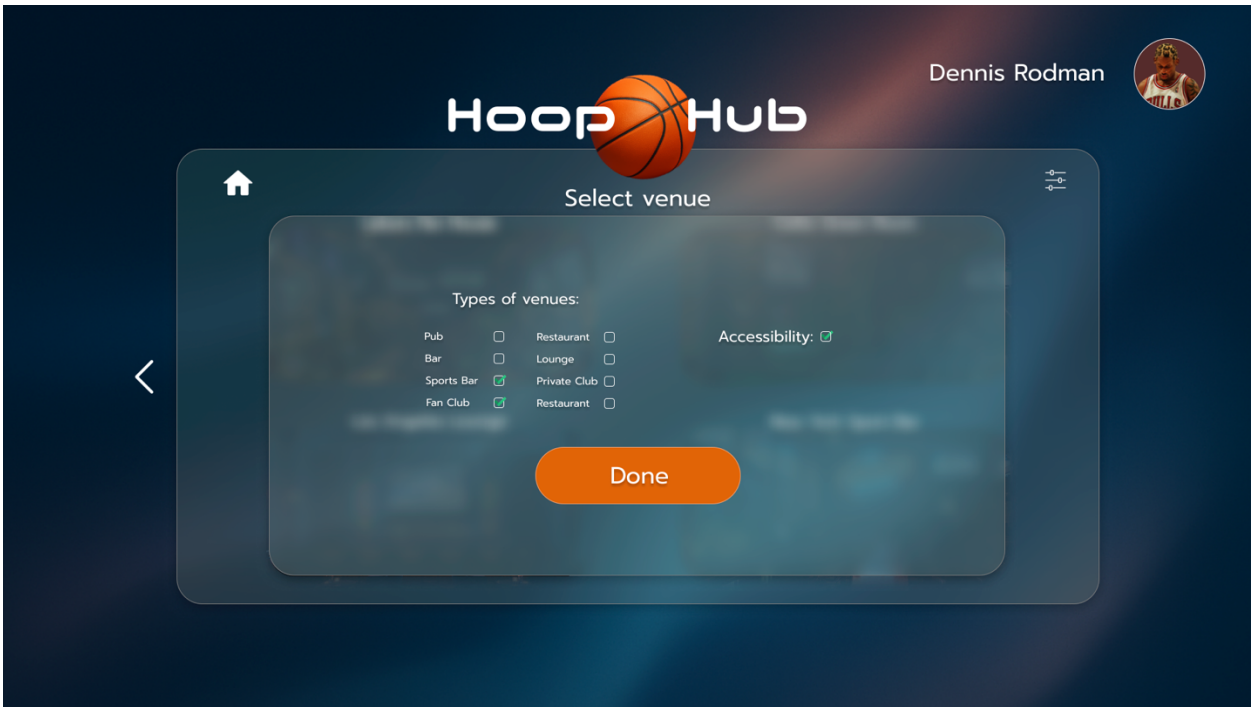
7a. *Selected venue is fully booked*: The System informs the Fan that the venue is full and offers the option to join the waitlist via use case *Join Waitlist**. Use case ends.

8a. *Fan cancel before confirming the booking request*: The System cancels the operation and returns the fan to their home page. Use case ends.

11a. *Venue Manager rejects the booking request*: The System updates status* to rejected and notifies the fan. Use case ends.

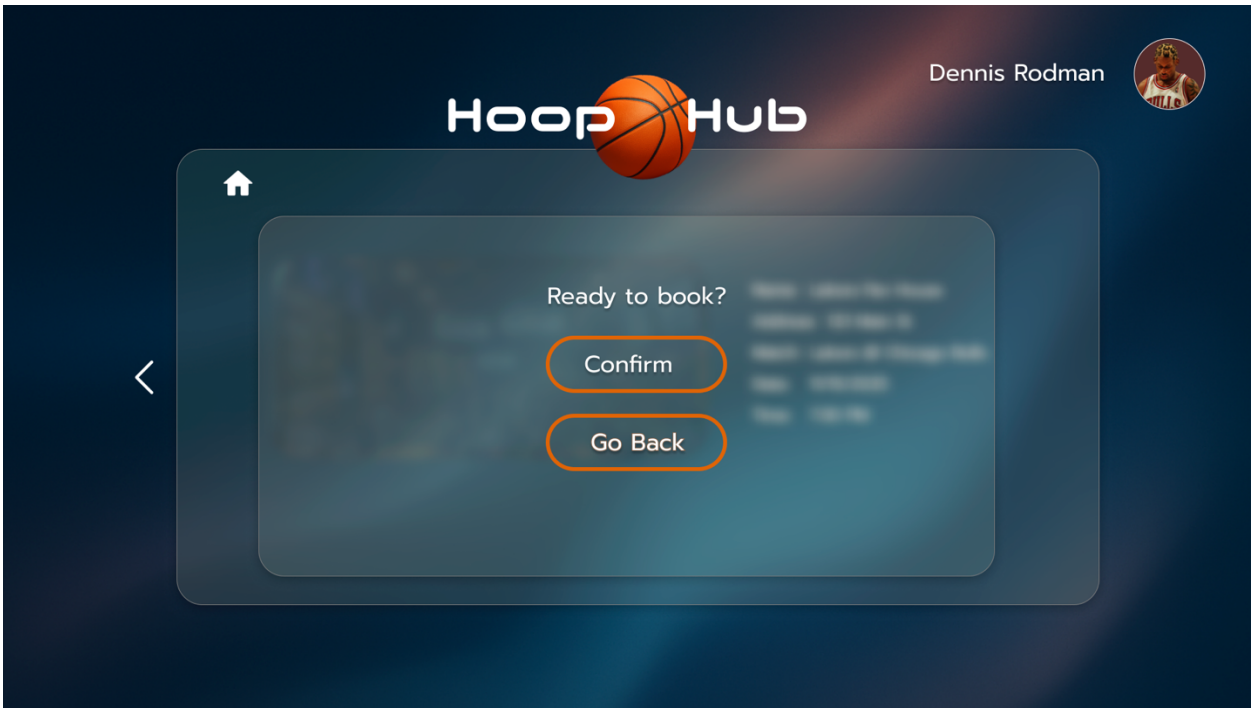
2 Storyboards

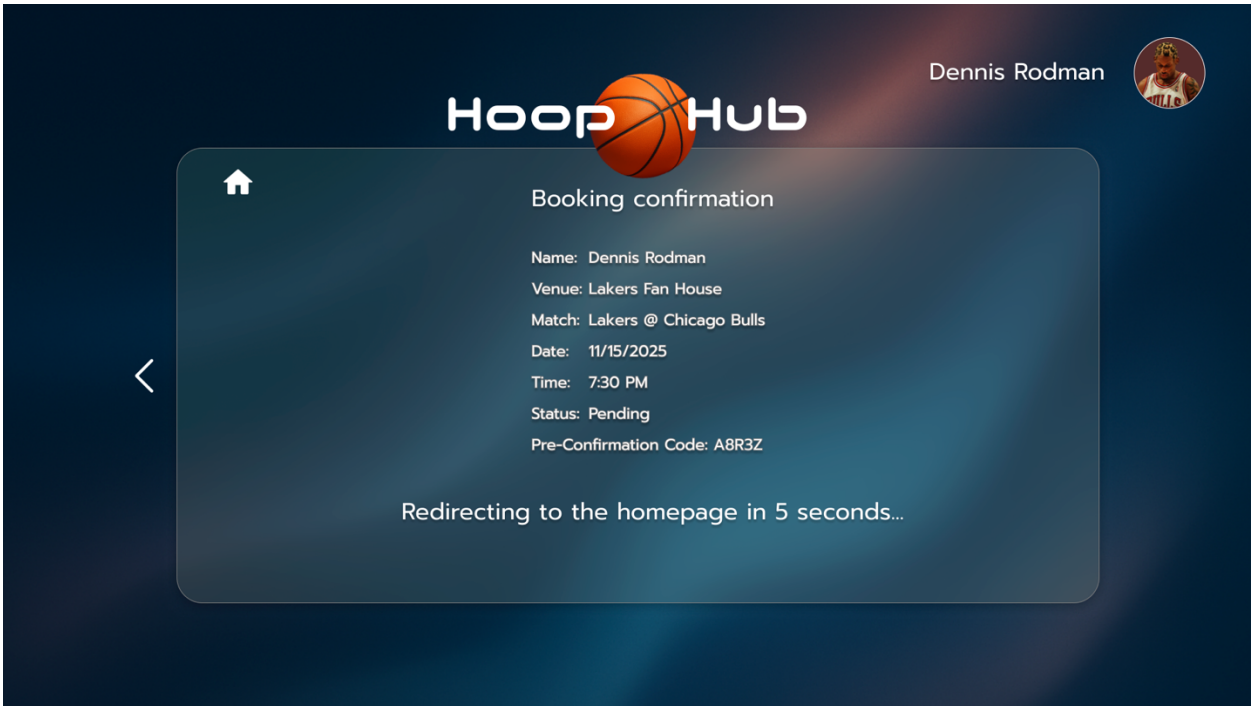

Dennis Rodman
Hoop Hub
Select venue
Lakers Fan House
Celtic Green Room
Los Angeles Lounge
New York Sport Bar


Dennis Rodman
Hoop Hub
Select venue
Types of venues:
Pub
Bar
Sports Bar
Fan Club
Restaurant
Lounge
Private Club
Restaurant
Accessibility:
Done


Dennis Rodman
Hoop Hub
Select venue
Lakers Fan House
Celtic Green Room
Miami Heat Faz Zone
New York Sport Bar

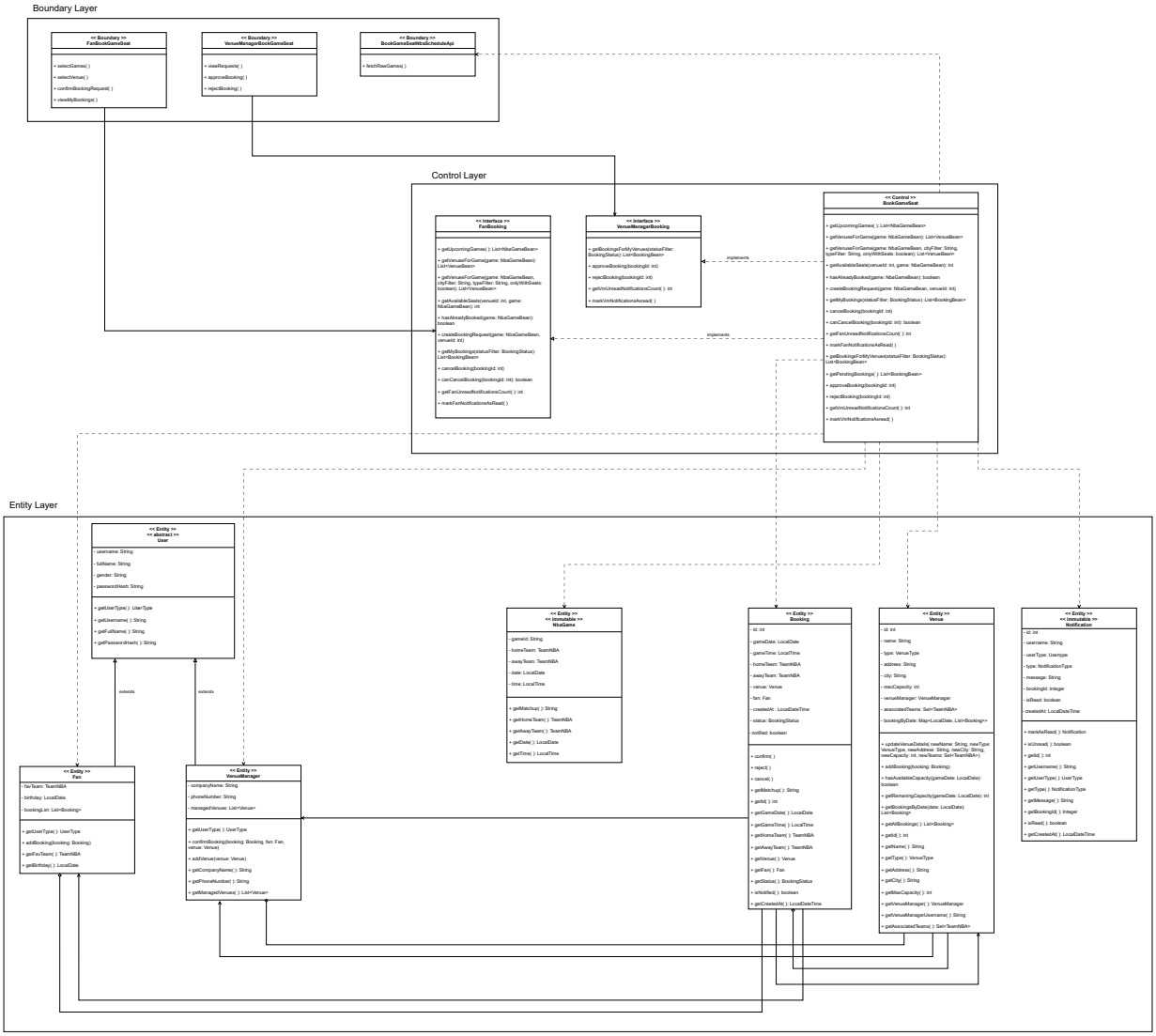

Dennis Rodman
Hoop Hub
Venue details
Name: Lakers Fan House
Address: 123 Main St
Match: Lakers @ Chicago Bulls
Date: 11/15/2025
Time: 7:30 PM
Confirm


Hoop Hub
Dennis Rodman
Ready to book?
Confirm
Go Back

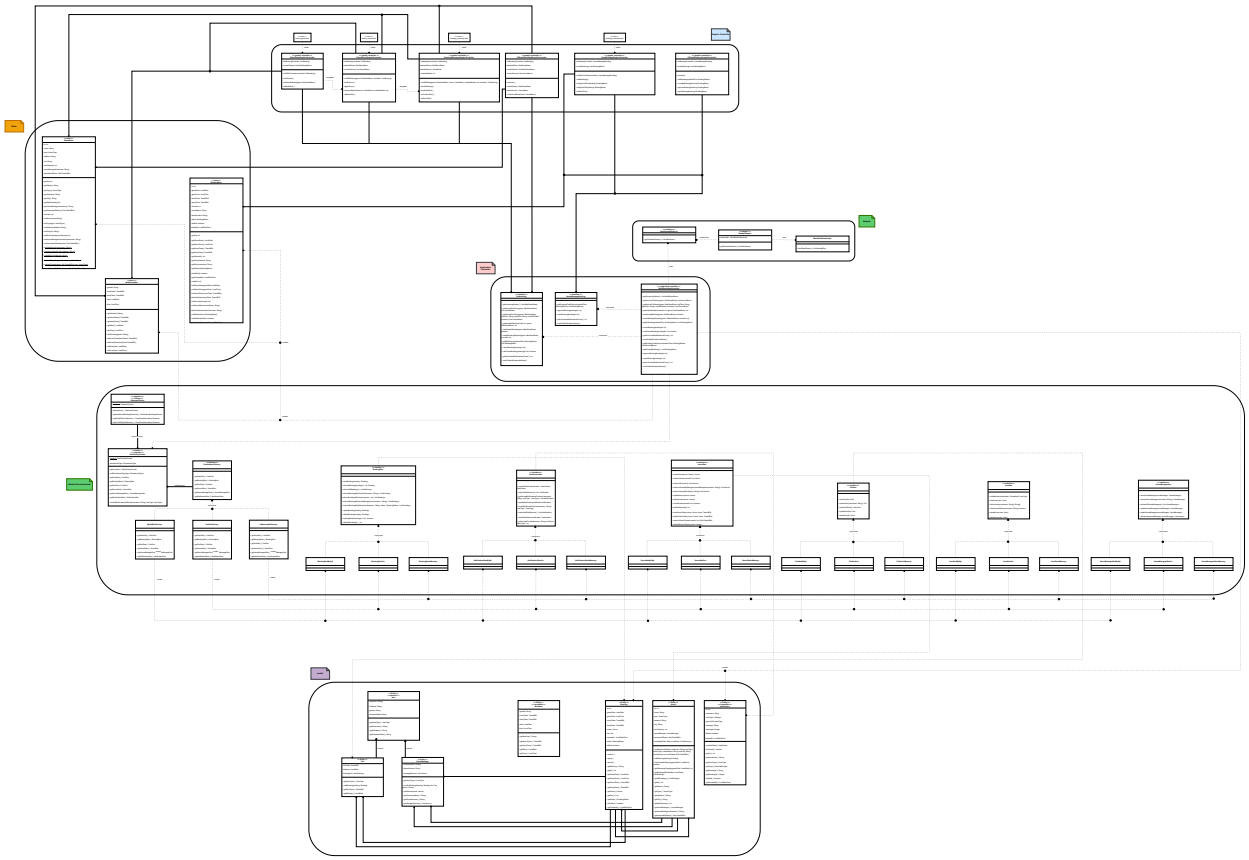

Hoop Hub
Dennis Rodman
Booking confirmation
Name: Dennis Rodman
Venue: Lakers Fan House
Match: Lakers @ Chicago Bulls
Date: 11/15/2025
Time: 7:30 PM
Status: Pending
Pre-Confirmation Code: A8R3Z
Redirecting to the homepage in 5 seconds...

3 Design

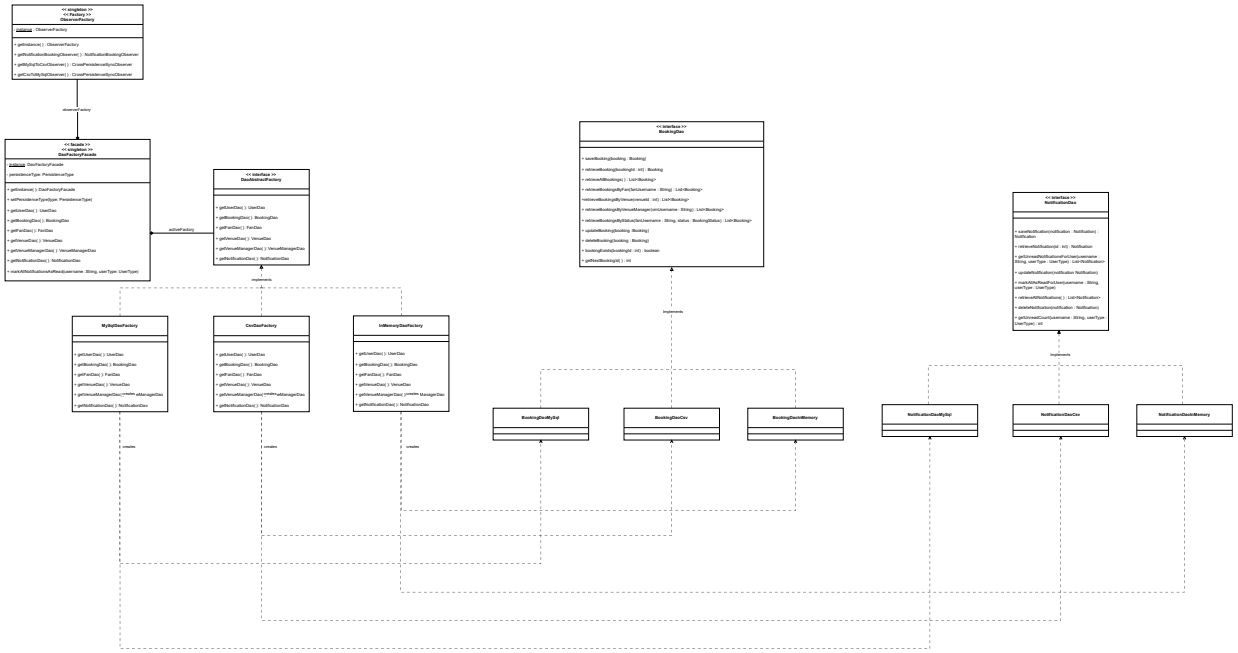
1. Class Diagram



2. Design-Level Diagram



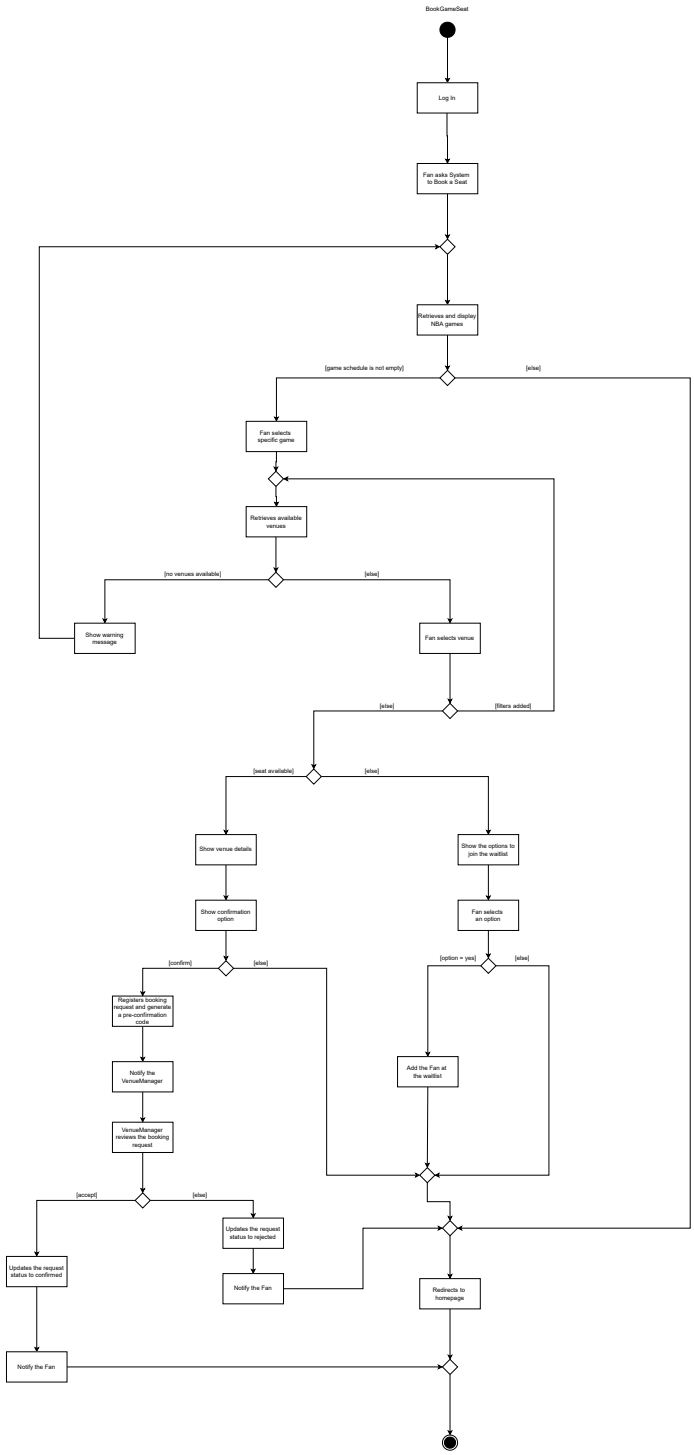
3. **Design Patterns**

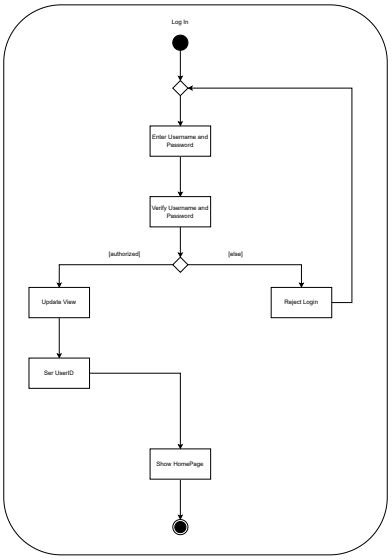


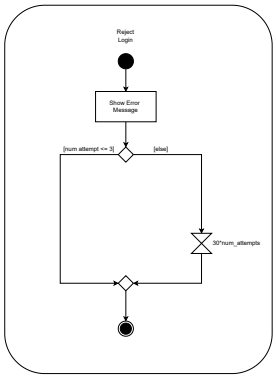
The diagram illustrates the architecture of the persistence layer, designed to ensure modularity, extensibility, and decoupling from implementation details. The solution integrates three fundamental GoF design patterns:

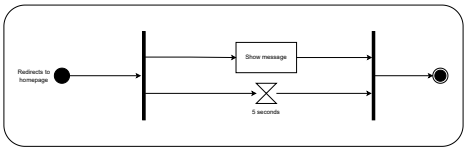
1. **Façade Pattern:** The DaoFactoryFacade class acts as the single point of access to the persistence layer. It hides from clients the complexity of connection management, persistence configuration (MySQL, CSV, In-Memory), and data synchronization.
2. **Abstract Factory Pattern:** To avoid scattered conditional statements and ensure consistency across the product family, the DaoAbstractFactory interface was introduced. The façade maintains a polymorphic reference (activeFactory) to one of the concrete implementations (MySqlDaoFactory, CsvDaoFactory, InMemoryDaoFactory), delegating the physical creation of DAOs to it. This way, adding a new persistence technology only requires creating a new Factory class without modifying existing code.
3. **Singleton Pattern:** Applied to both DaoFactoryFacade and ObserverFactory to ensure that shared resources (DAO cache and Observer instances) are centralized and unique within the application.

4. **Activity Diagram**

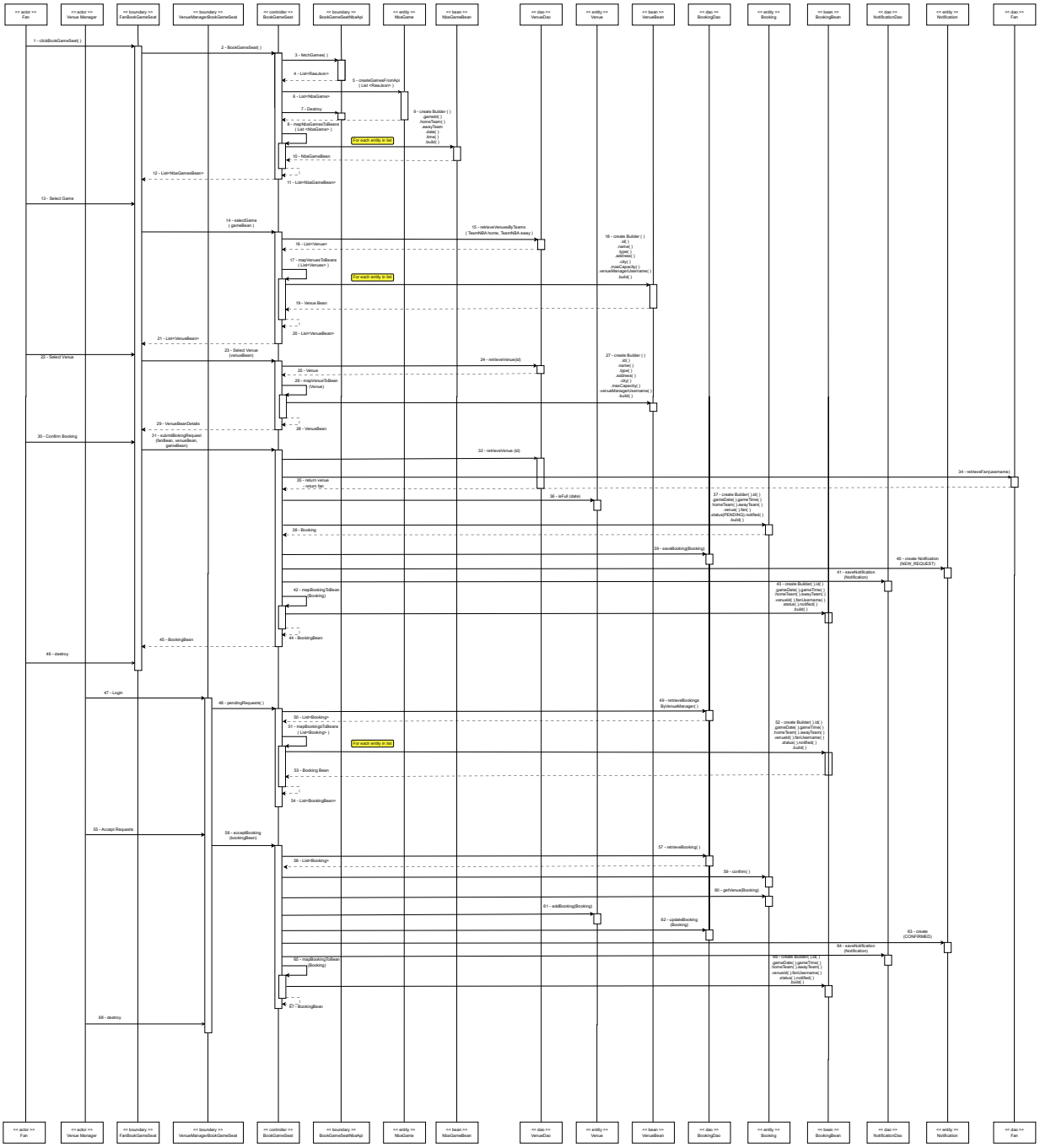




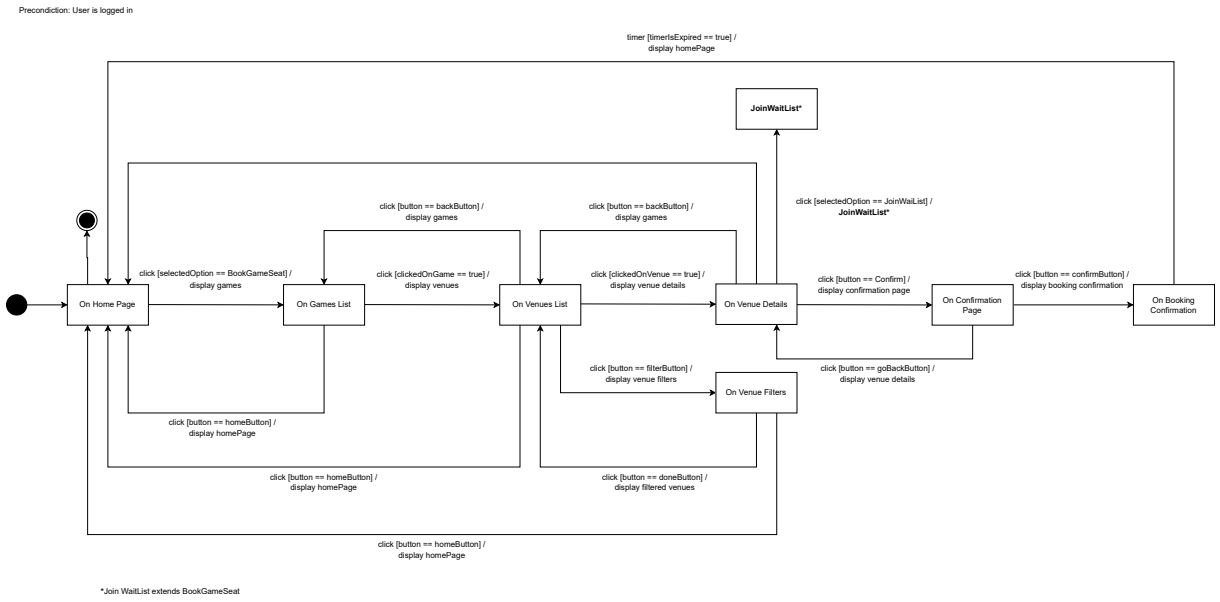




5. **Sequence Diagram**



6. State Diagram



4 Testing

TestBookingSyncIntegration

1. Verify that a booking saved on CSV is synchronized to MySQL.
2. Verify that the source booking on CSV remains intact after synchronization.
3. Verify that a booking saved on MySQL is synchronized to CSV.
4. Verify that the status of the booking synchronized to CSV is correct.
5. Verify that the fan associated with the booking synchronized to CSV is correct.
6. Verify that the date of the booking synchronized to CSV is correct.
7. Verify that UPSERT does not generate duplication errors on an already existing ID.
8. Verify that update to CONFIRMED persists correctly on CSV.
9. Verify that update to CANCELLED persists correctly on CSV.
10. Verify that synchronization does not generate infinite loops.
11. Verify that bookings data remains intact after loop prevention.
12. Verify bidirectional synchronization: both bookings are present on CSV.

5 GitHub

Link to the GitHub repository.

6 SonarQube Cloud

Link to the project on SonarQube Cloud.

7 Demo Video

Link to the demo video.